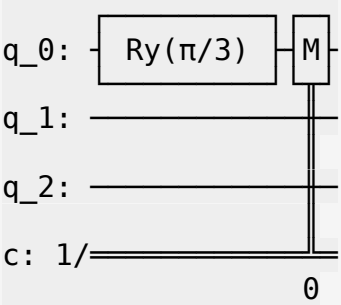
```
%pip install qiskit
%pip install qiskit_aer
%pip install numpy
%pip install matplotlib
%pip install pylatexenc
```

```
Requirement already satisfied: qiskit in
/usr/local/lib/python3.11/dist-packages (1.3.2)
Requirement already satisfied: rustworkx>=0.15.0 in
/usr/local/lib/python3.11/dist-packages (from qiskit) (0.16.0)
Requirement already satisfied: numpy<3,>=1.17 in
/usr/local/lib/python3.11/dist-packages (from qiskit) (1.26.4)
Requirement already satisfied: scipy>=1.5 in
/usr/local/lib/python3.11/dist-packages (from qiskit) (1.13.1)
Requirement already satisfied: sympy>=1.3 in
/usr/local/lib/python3.11/dist-packages (from qiskit) (1.13.1)
Requirement already satisfied: dill>=0.3 in
/usr/local/lib/python3.11/dist-packages (from qiskit) (0.3.9)
Requirement already satisfied: python-dateutil>=2.8.0 in
/usr/local/lib/python3.11/dist-packages (from qiskit) (2.8.2)
Requirement already satisfied: stevedore>=3.0.0 in
/usr/local/lib/python3.11/dist-packages (from qiskit) (5.4.0)
Requirement already satisfied: typing-extensions in
/usr/local/lib/python3.11/dist-packages (from qiskit) (4.12.2)
Requirement already satisfied: symengine<0.14,>=0.11 in
/usr/local/lib/python3.11/dist-packages (from qiskit) (0.13.0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.0-
>qiskit) (1.17.0)
Requirement already satisfied: pbr>=2.0.0 in
/usr/local/lib/python3.11/dist-packages (from stevedore>=3.0.0-
>qiskit) (6.1.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.11/dist-packages (from sympy>=1.3->qiskit)
(1.3.0)
Requirement already satisfied: qiskit_aer in
/usr/local/lib/python3.11/dist-packages (0.16.0)
Requirement already satisfied: qiskit>=1.1.0 in
/usr/local/lib/python3.11/dist-packages (from qiskit_aer) (1.3.2)
Requirement already satisfied: numpy>=1.16.3 in
/usr/local/lib/python3.11/dist-packages (from qiskit_aer) (1.26.4)
Requirement already satisfied: scipy>=1.0 in
/usr/local/lib/python3.11/dist-packages (from qiskit_aer) (1.13.1)
Requirement already satisfied: psutil>=5 in
/usr/local/lib/python3.11/dist-packages (from qiskit_aer) (5.9.5)
Requirement already satisfied: rustworkx>=0.15.0 in
/usr/local/lib/python3.11/dist-packages (from qiskit>=1.1.0-
>qiskit_aer) (0.16.0)
Requirement already satisfied: sympy>=1.3 in
/usr/local/lib/python3.11/dist-packages (from qiskit>=1.1.0-
```

```
>qiskit_aer) (1.13.1)
Requirement already satisfied: dill>=0.3 in
/usr/local/lib/python3.11/dist-packages (from qiskit>=1.1.0-
>qiskit_aer) (0.3.9)
Requirement already satisfied: python-dateutil>=2.8.0 in
/usr/local/lib/python3.11/dist-packages (from qiskit>=1.1.0-
>qiskit_aer) (2.8.2)
Requirement already satisfied: stevedore>=3.0.0 in
/usr/local/lib/python3.11/dist-packages (from qiskit>=1.1.0-
>qiskit_aer) (5.4.0)
Requirement already satisfied: typing-extensions in
/usr/local/lib/python3.11/dist-packages (from qiskit>=1.1.0-
>qiskit_aer) (4.12.2)
Requirement already satisfied: symengine<0.14,>=0.11 in
/usr/local/lib/python3.11/dist-packages (from qiskit>=1.1.0-
>qiskit_aer) (0.13.0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.0-
>qiskit>=1.1.0->qiskit_aer) (1.17.0)
Requirement already satisfied: pbr>=2.0.0 in
/usr/local/lib/python3.11/dist-packages (from stevedore>=3.0.0-
>qiskit>=1.1.0->qiskit_aer) (6.1.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.11/dist-packages (from sympy>=1.3-
>qiskit>=1.1.0->qiskit_aer) (1.3.0)
Requirement already satisfied: numpy in
/usr/local/lib/python3.11/dist-packages (1.26.4)
Requirement already satisfied: matplotlib in
/usr/local/lib/python3.11/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (1.3.1)
Requirement already satisfied: cycler>=0.10 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (4.55.7)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (1.4.8)
Requirement already satisfied: numpy>=1.23 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (1.26.4)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.11/dist-packages (from matplotlib) (2.8.2)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.7-
```

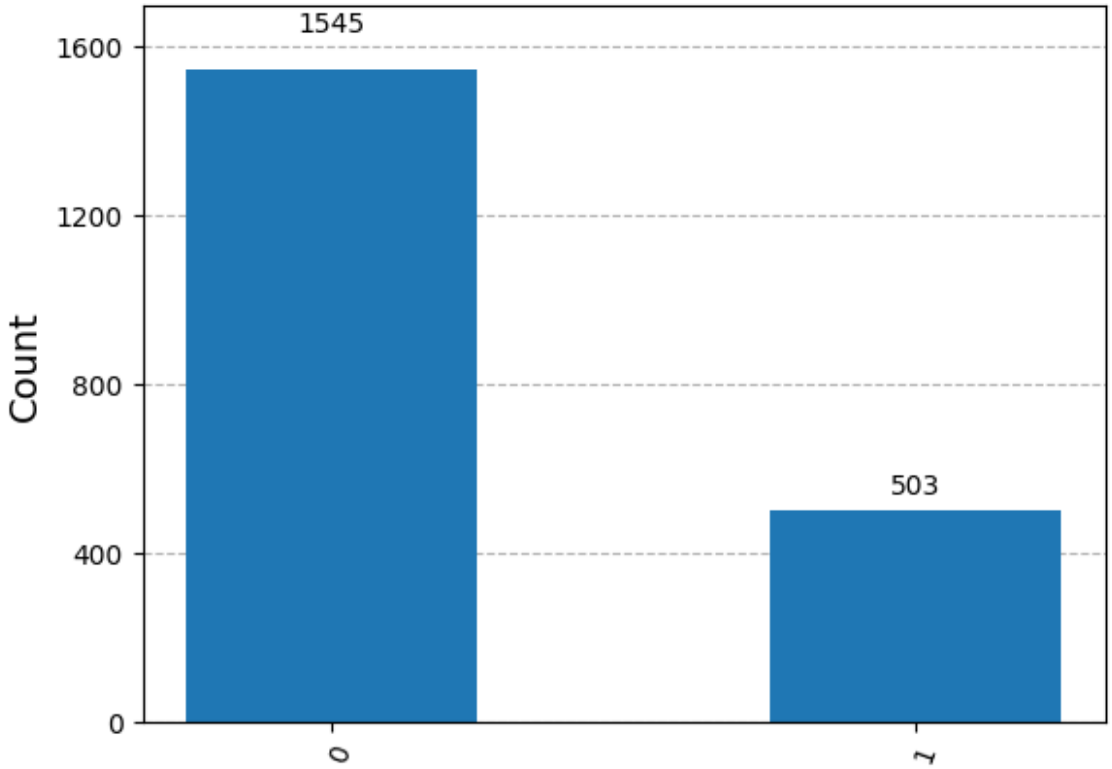
```
>matplotlib) (1.17.0)
Requirement already satisfied: pylatexenc in
/usr/local/lib/python3.11/dist-packages (2.10)

%matplotlib inline

from qiskit import QuantumCircuit, QuantumRegister
from qiskit.quantum_info import Statevector, random_unitary
from qiskit.circuit.library.data_preparation import Initialize
from qiskit.visualization import plot_histogram
import numpy as np
from qiskit.circuit.library import IGate, XGate, ZGate

from qiskit_aer import Aer, AerSimulator
from qiskit.quantum_info import partial_trace
backend = AerSimulator(method='statevector')

# EXERCICIO 1 (Preparação de estado)
qc = QuantumCircuit(3, 1)

qc.ry(np.pi/3, 0)
qc.measure(0, 0)

qc.draw()
```




```
# Verificar que há 75% de probabilidade de medir |0> no primeiro qubit
job = backend.run(qc, shots=2048)
result = job.result()
counts = result.get_counts()
print(counts)

# Contar as ocorrências de |0> no primeiro qubit
num_zeros = sum(count for state, count in counts.items() if state[-1]
== '0')
total_shots = sum(counts.values())
percent_zeros = (num_zeros / total_shots) * 100

# Exibir os resultados
print(f"Percentagem de medições |0⟩ no primeiro qubit:
{percent_zeros:.2f}%")
```

```
plot_histogram(counts)
```

```
{'1': 503, '0': 1545}
Percentagem de medições |0⟩ no primeiro qubit: 75.44%
```

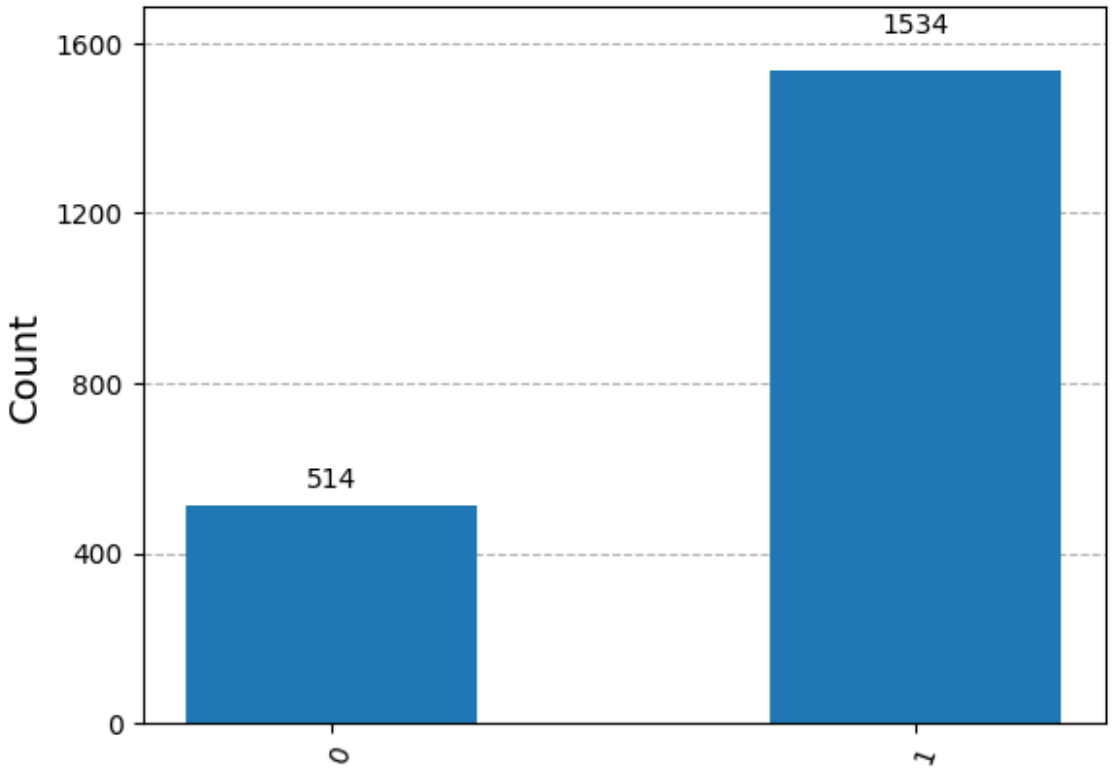



```
#EXERCICIO 2 (Erro do tipo Y)
# Circuito igual mas com a operação Y no estado final
qc = QuantumCircuit(3, 1)

qc.ry(np.pi/3, 0)

qc.y(0)
qc.measure(0, 0)

qc.draw()
```

```
c: 1/═══════════════════╩═
                        0
```

```python
# Verificar que há 75% de probabilidade de medir |1> no primeiro qubit
job = backend.run(qc, shots=2048)
result = job.result()
counts = result.get_counts()
print(counts)

# Contar as ocorrências de |0> e |1> no primeiro qubit
num_zeros = sum(count for state, count in counts.items() if state[-1]
== '0')
total_shots = sum(counts.values())
percent_zeros = (num_zeros / total_shots) * 100
num_ones = sum(count for state, count in counts.items() if state[-1]
== '1')
percent_ones = (num_ones / total_shots) * 100


# Exibir os resultados
print(f"Percentagem de medições |0⟩ no primeiro qubit:
{percent_zeros:.2f}%")
print(f"Percentagem de medições |1⟩ no primeiro qubit:
{percent_ones:.2f}%")

plot_histogram(counts)
```

```
{'0': 514, '1': 1534}
Percentagem de medições |0⟩ no primeiro qubit: 25.10%
Percentagem de medições |1⟩ no primeiro qubit: 74.90%
```



Após a introdução do erro Y as probabilidades de medir o quibit no estado

$$P(|0\rangle)=0.25$$

Para proteger contra erros do tipo Y, vamos usar a relação

$$S^{\dagger} Y S= X$$

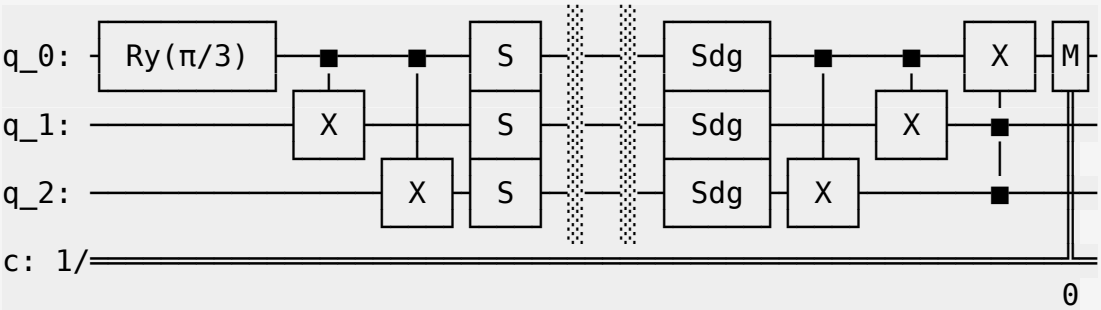
sendo S a operação associada à matriz:

$$S=\begin{pmatrix}1 & 0\\ 0 & i\end{pmatrix}$$

e sendo i a unidade imaginária.

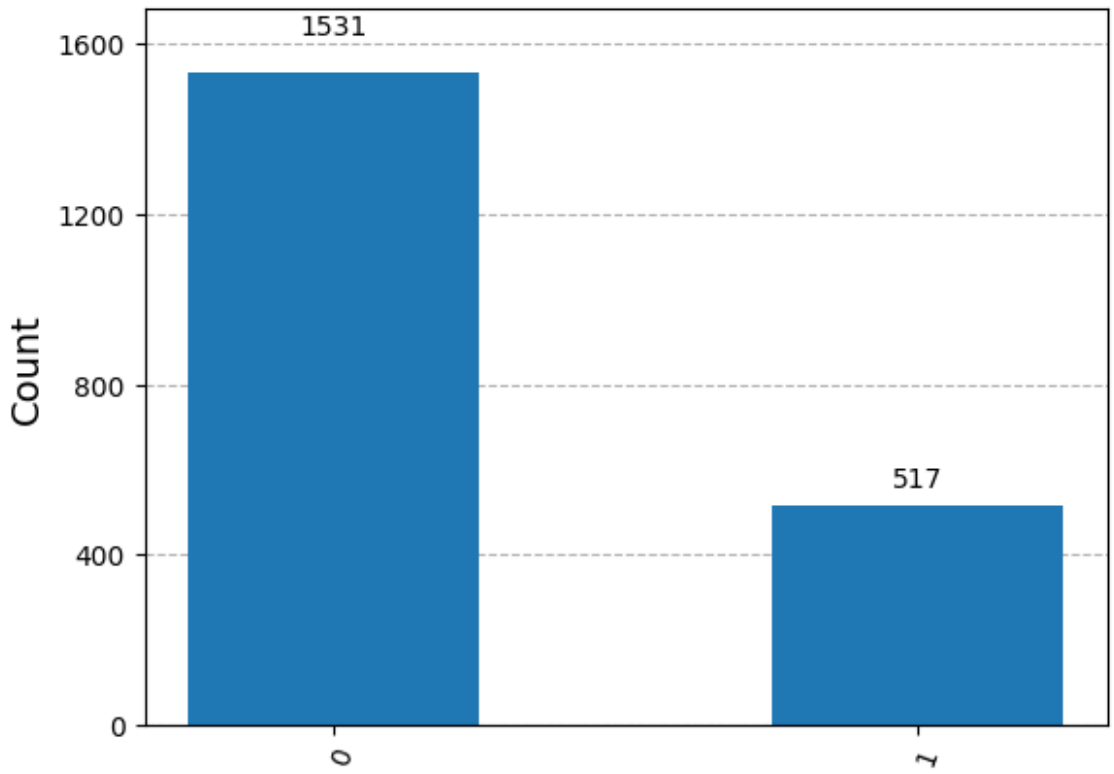
```
#EXERCICIO 3 (Correção de erros)
def erro_y(qubit):
    qc = QuantumCircuit(3, 1)
    qc.ry(np.pi/3, 0)
    qc.cx(0, 1)
    qc.cx(0, 2)

    qc.s(range(3))

    qc.barrier()
    if(qubit != -1):
        qc.y(qubit)
```

```
    qc.barrier()

    qc.sdg(range(3))

    qc.cx(0, 2)
    qc.cx(0, 1)
    qc.ccx(2, 1, 0)

    qc.measure(0, 0)

    return qc

#(a)
qc = erro_y(-1)

#circuito
qc.draw()
```

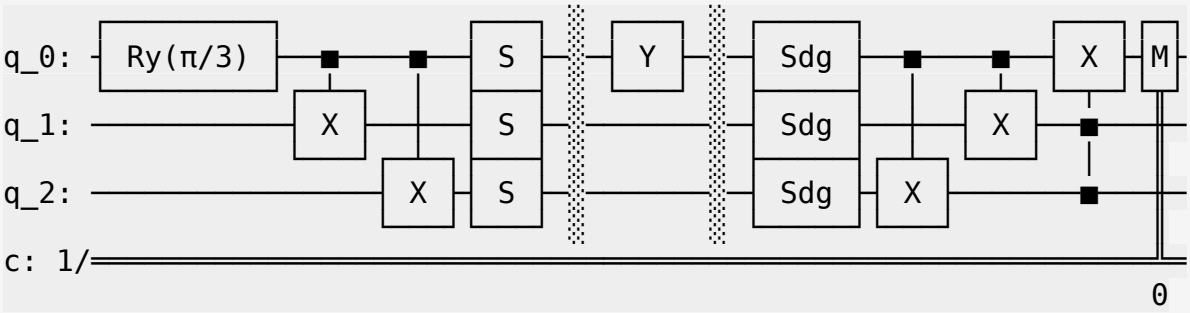



```
job = backend.run(qc, shots=2048)
result = job.result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

```
{'1': 517, '0': 1531}
```

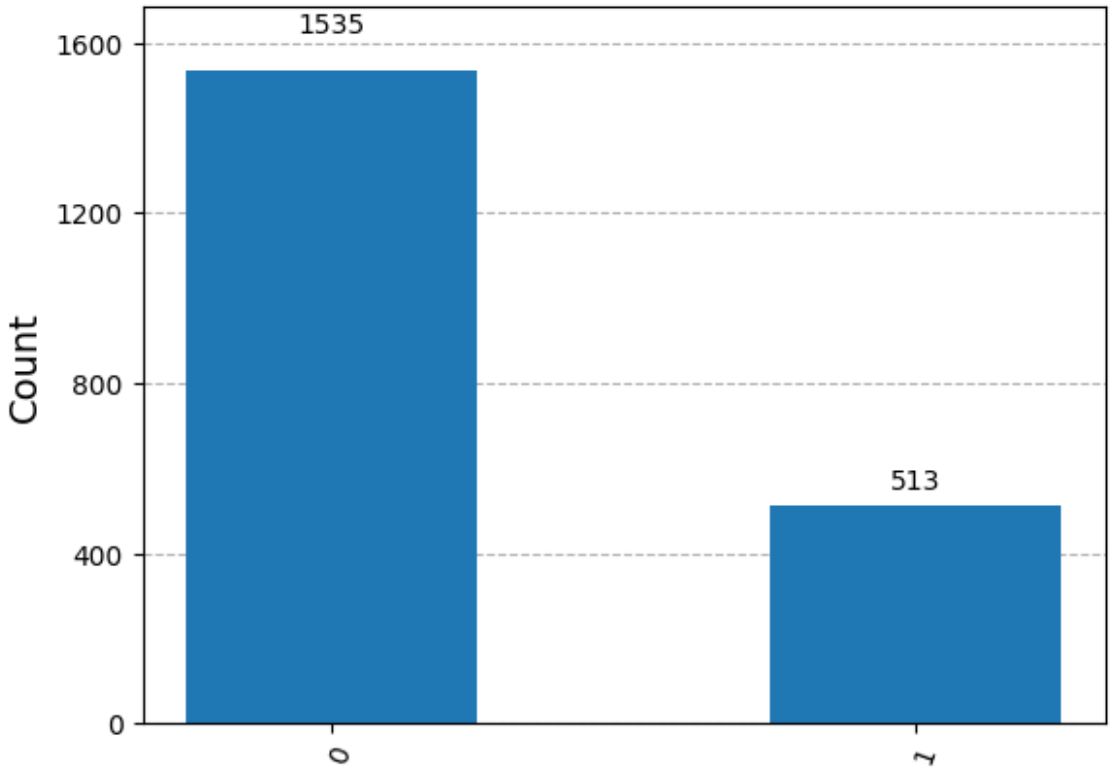



```
# 3(b)

qc = erro_y(0)
qc.draw()
```

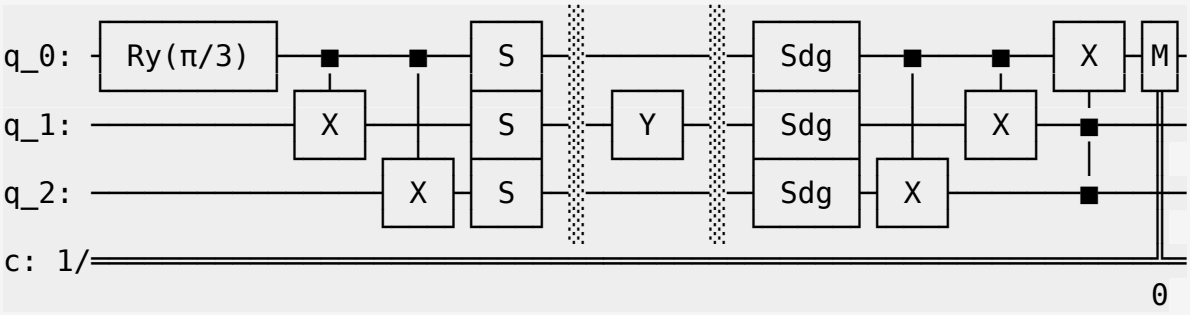



```
job = backend.run(qc, shots=2048)
result = job.result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

```
{'1': 513, '0': 1535}
```

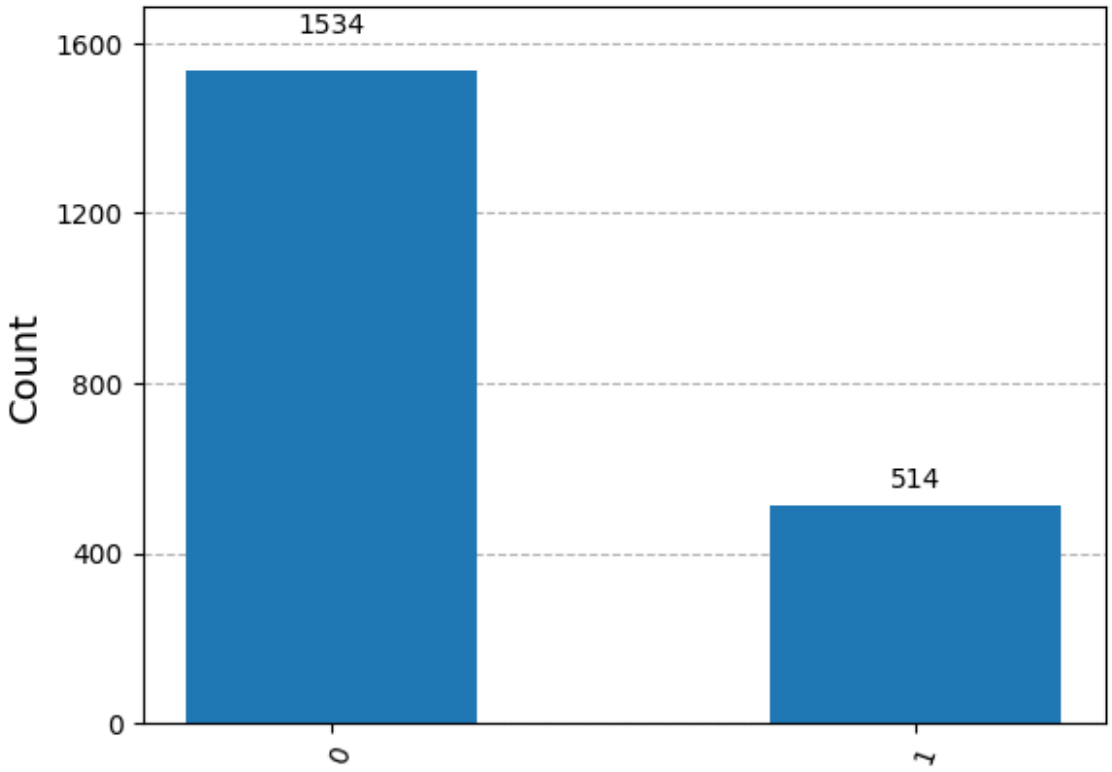



```
# 3 (c)
qc = erro_y(1)
qc.draw()
```

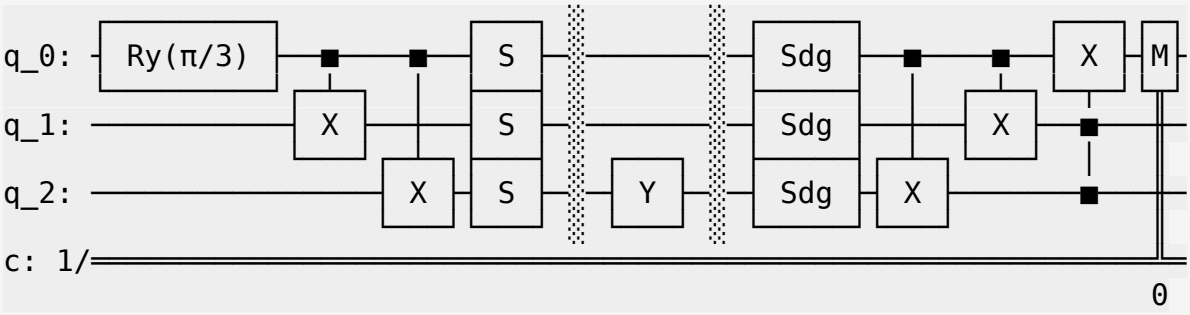



```
job = backend.run(qc, shots=2048)
result = job.result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

```
{'1': 514, '0': 1534}
```

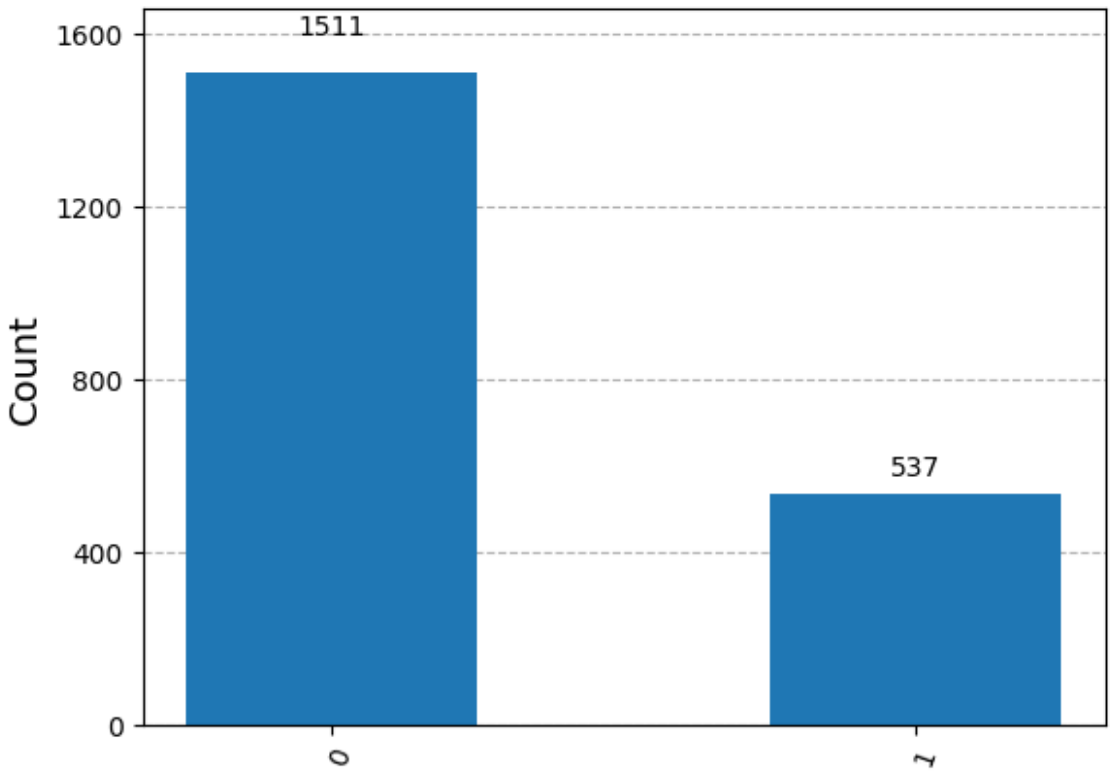



```
# 3 (d)
qc = erro_y(2)
qc.draw()
```




```
job = backend.run(qc, shots=2048)
result = job.result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

```
{'1': 537, '0': 1511}
```



Devido à inalteração das medições verifica-se que os erros do tipo Y estão a ser corrigidos

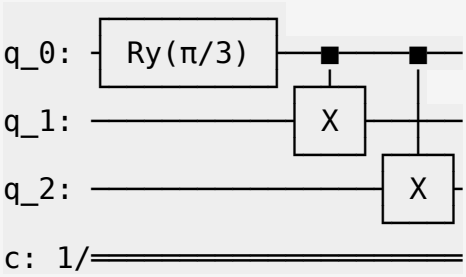
```
def formata(z):
    if abs(z) < 1e-10: return "0"
    if z.imag == 0: return f"{z.real:.3f}"
    if z.real == 0: return f"{z.imag:.3f}i"
    return f"{z.real:.3f}{'+' if z.imag >= 0 else ''}{z.imag:.3f}i"

def estado(qc, step_name=""):
    backend = Aer.get_backend('statevector_simulator')
    job = backend.run(qc)
    result = job.result()
    sv = np.asarray(result.get_statevector())

    print(f"\n{step_name}")

    state = "|ψ⟩ = "
    n_qubits = len(qc.qubits)

    for i, amp in enumerate(sv):
        if abs(amp) > 1e-10:
            basis = format(i, f'0{n_qubits}b')
            if state != "|ψ⟩ = ":
                state += " + " if amp.real >= 0 else " - "
            state += f"({formata(abs(amp) if amp.real < 0 else amp)})|
{basis}⟩"
```

```
    print(state)
    print("\nProbabilidades de medição:")
    for i, amp in enumerate(sv):
        prob = abs(amp)**2
        if prob > 1e-10:
            basis = format(i, f'0{n_qubits}b')
            print(f"P(|{basis}⟩) = {prob:.3f}")
```

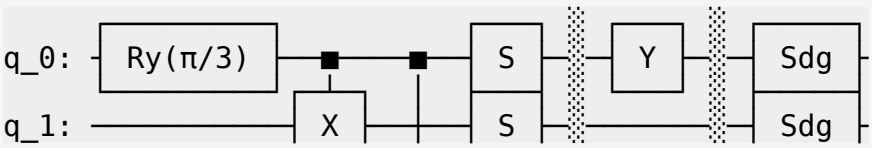
Diferentes etapas do circuito:

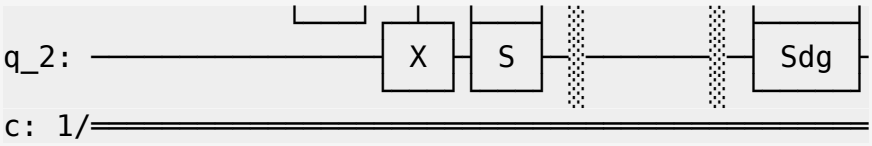
```
#etapa 1:

qc = QuantumCircuit(3, 1)
qc.ry(np.pi/3, 0)

estado(qc, "Inicial")
qc.draw()

# Como verificado no primeiro exercício, após aplicar uma rotação em Y de pi/3, a probabilidade de medir o estado |0> no primeiro quibit é de 75%


Inicial
|ψ⟩ = (0.866)|000⟩ + (0.500)|001⟩

Probabilidades de medição:
P(|000⟩) = 0.750
P(|001⟩) = 0.250

          ┌──────────┐
q_0: ─────┤ Ry(π/3)  ├──
          └──────────┘
q_1: ─────────────────────

q_2: ─────────────────────

c: 1/═════════════════════


#etapa2
qc.cx(0, 1)
qc.cx(0, 2)

estado(qc, "Com CNOTs")
qc.draw()

#Como o primeiro qubit é o qubit de controle das operações CNOT, o seu
```

```
estado afeta os outros dois qubits. A probabilidade deste ser |1> é
passada para os outros 2 qubits.


Com CNOTs
|ψ⟩ = (0.866)|000⟩ + (0.500)|111⟩

Probabilidades de medição:
P(|000⟩) = 0.750
P(|111⟩) = 0.250
```



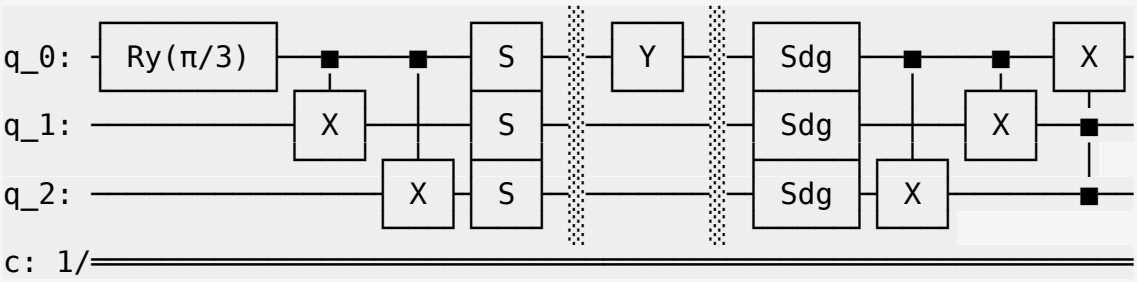

```
#etapa3
qc.s(range(3))
qc.barrier()
qc.y(0)
qc.barrier()
qc.sdg(range(3))

estado(qc, "Após gates S")
qc.draw()

#Sabemos que Sdg*Y*S = X e que Sdg*S = I. Portanto, no qubit onde o
erro foi aplicado, esse erro fica a ser um erro de X (ou seja, um bit-
flip).
#Como resultado, a probabilidade de medir |1⟩ no primeiro qubit passa
a ser 75%. Nos demais qubits, o estado permanece inalterado.

# Este tipo de erro (bit-flip) é corrigível com o código de Shor:


Após gates S
|ψ⟩ = (0.866)|001⟩ + (0.500)|110⟩

Probabilidades de medição:
P(|001⟩) = 0.750
P(|110⟩) = 0.250
```






```
#ultima etapa
qc.cx(0, 2)
qc.cx(0, 1)
qc.ccx(2, 1, 0)
estado(qc, "Após CNOTs finais")
qc.draw()

# Por fim aplicamos os CNOTs para desfazer a operação inicial de
controlo,
# Isto resulta num estado no qual a probabilidade de medir |0> no
primeiro qubit volta a ser 75%
```

```
Após CNOTs finais
|ψ⟩ = (0.866)|110⟩ + (0.500)|111⟩

Probabilidades de medição:
P(|110⟩) = 0.750
P(|111⟩) = 0.250
```



Deste modo e com estes circuitos, o erro de Y no qubit é corrigido. O qubit de controlo é utilizado para propagar o erro para os outros qubits, e, em seguida, os outros qubits são usados para corrigir o erro no qubit original de controlo.

Isto funciona desde que o erro não ocorra em mais do que 1 qubit ao mesmo tempo. O processo baseia-se num majority vote (a decisão final sobre o estado de um qubit é tomada com base no estado que ocorre com maior frequência entre vários qubits) para determinar o estado correto.

```
#EXERCICIO 4 (Erros genéricos)

#circuito para corrigir erros genéricos, não só erros de Y, mas também
erros de X e Z

# Erro X: |0> -> |1> e |1> -> |0>
```

```
# Erro Z: |+> -> |-> e |-> -> |+>
# Erro Y: |i> -> -|i> e |-i> -> -|-i> (Y = iXZ)

qc = QuantumCircuit(3, 1)
qc.ry(np.pi/3, 0)
qc.cx(0, 1)
qc.cx(0, 2)

qc.s(range(3))

qc.barrier()
qc.y(0)
qc.barrier()

qc.sdg(range(3))

qc.cx(0, 2)
qc.cx(0, 1)
qc.ccx(2, 1, 0)

qc.barrier()

# Medir o qubit 0
qc.measure(0, 0)

# Desenhar o circuito
qc.draw()
```